

AWS API Gateway Lab Part 2: API Using Lambda

To Begin with the Lab

Summary of the Lab

In this part of the lab, three **AWS Lambda** functions were created to handle the backend CRUD logic for **Amazon DynamoDB**. The first function inserted a new user record into the **UsersTable**, the second function fetched a user record by **userId**, and the third function deleted a user record by **userId**. Each Lambda function was created from scratch and attached to the previously created **LambdaDynamoDBExecutionRole** so it could access **DynamoDB**. The functions were then tested directly from the Lambda console using API Gateway–style JSON events, and the results were verified in **DynamoDB**. This completed the backend API logic that will later be exposed through **API Gateway**.

Understanding Lambda-Based API Logic

AWS Lambda is used here as the place where API logic runs behind the scenes. It exists so you can separate your application into small, focused operations such as creating a record, reading a record, and deleting a record. In this lab, each Lambda function handles exactly one task, which follows the separation of concerns principle and makes the code easier to test and maintain. The Lambda functions also need an **IAM role** so they can securely talk to **Amazon DynamoDB**.

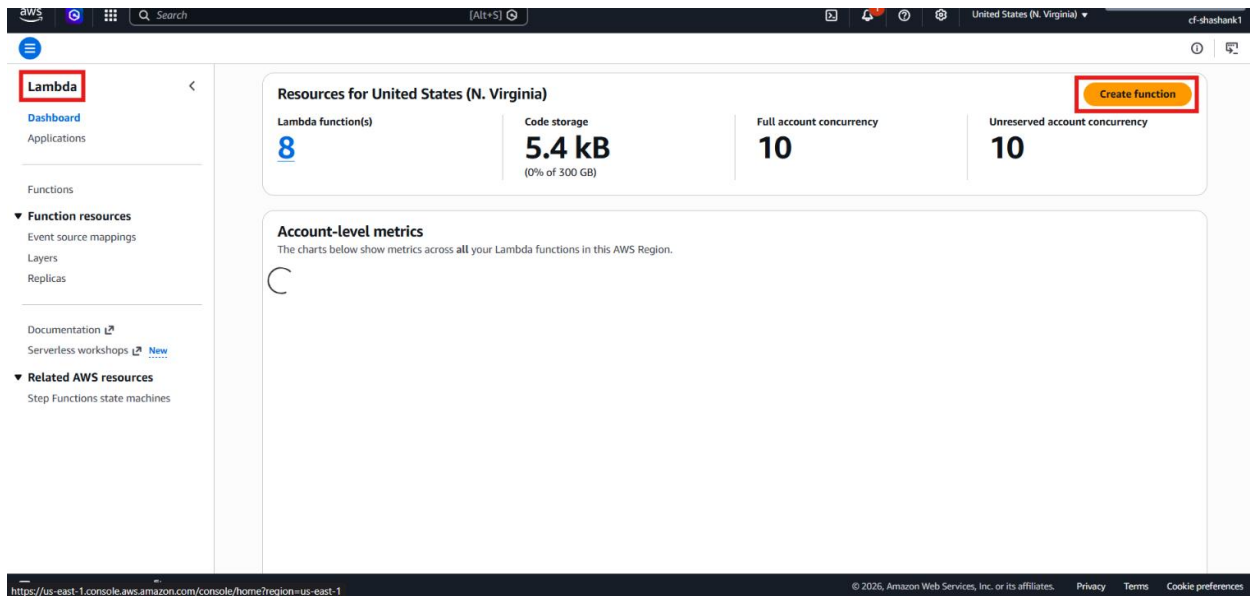
Example:

A mobile app sends a request to create a user named Bhavesh with **userId** 101. The **createUserFunction** receives the request, writes the record to **UsersTable**, and returns a success message. Later, **getUserFunction** can fetch the same record, and **deleteUserFunction** can remove it by the same **userId**.

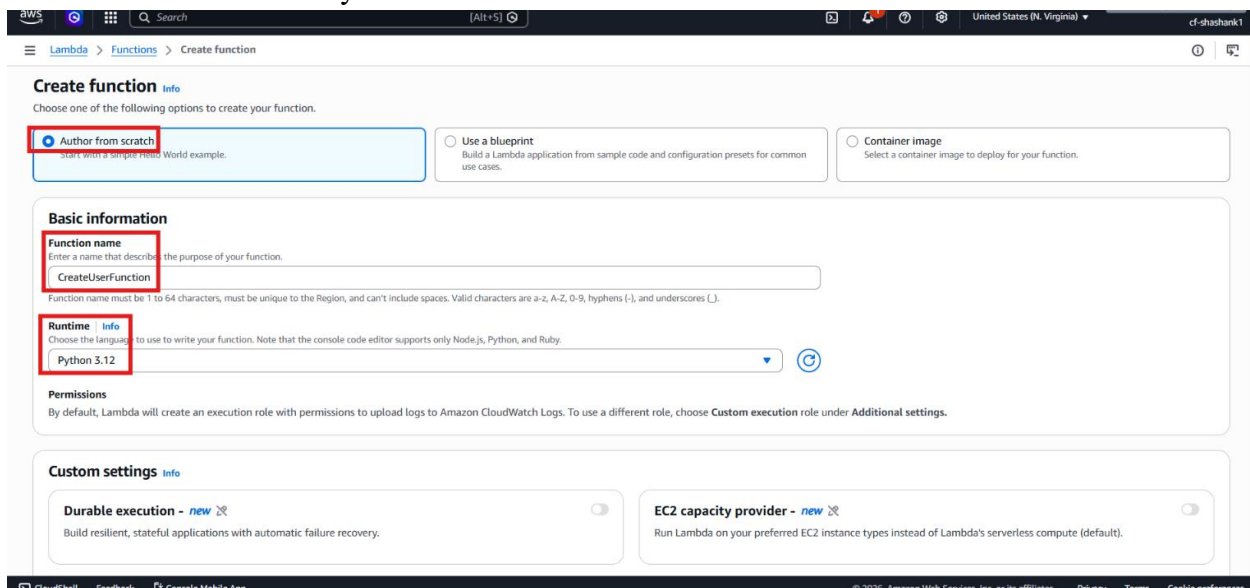
Lab Steps

Step 1 – Create the First Lambda Function

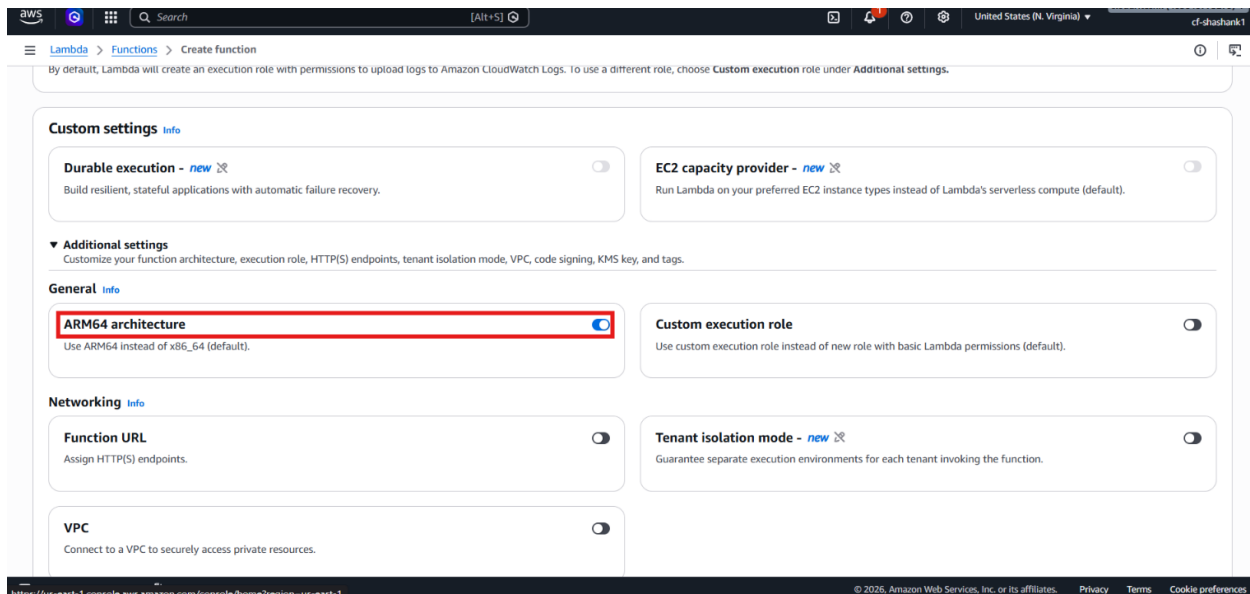
- Sign in to the AWS Management Console.
- Open **AWS Lambda**.
- Click **Create function**.



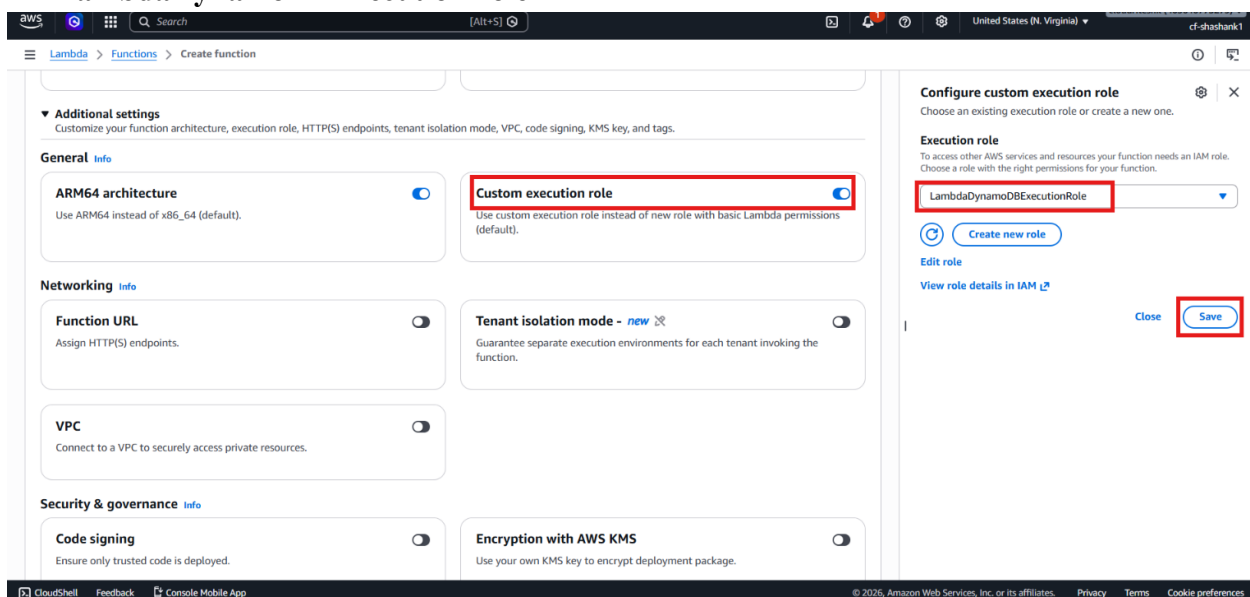
- Select **Author from scratch**.
- Enter the function name: **CreateUserFunction**
- Choose the runtime: **Python 3.12**



- Under Custom Settings → Additional Settings, enable **ARM64 architecture** to use **x86_64**.



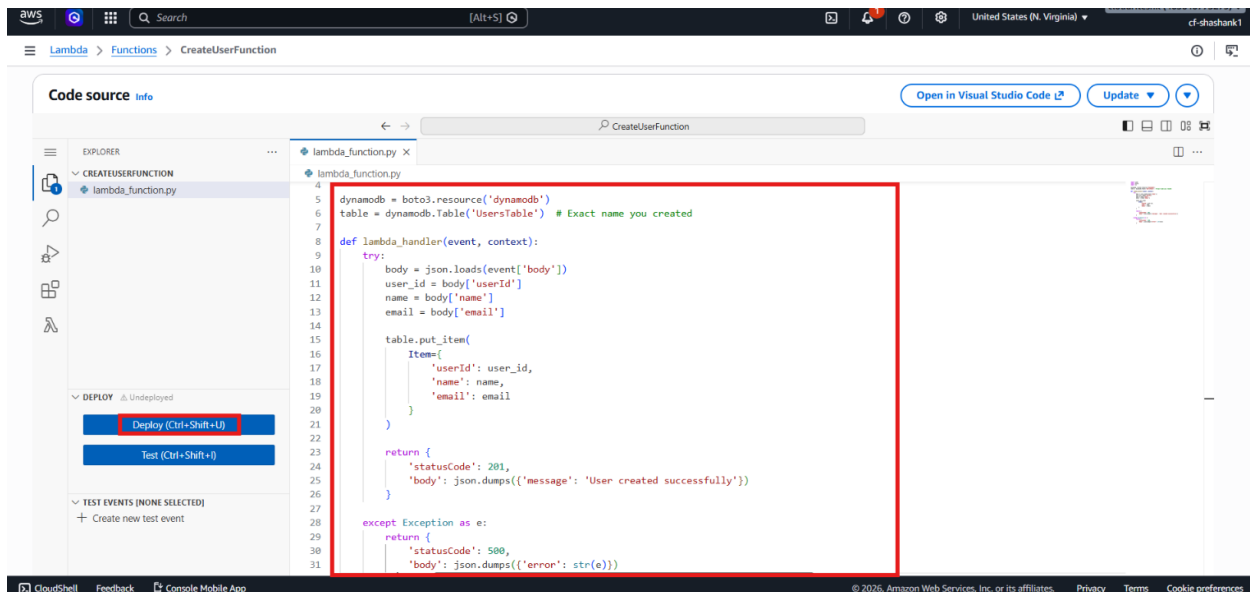
- Here Under **Additional Settings** click on **Custom Execution Role** and Select **LambdaDynamoDBExecutionRole**.



- Click **Create function**.

Step 2 – Add the Create User Code

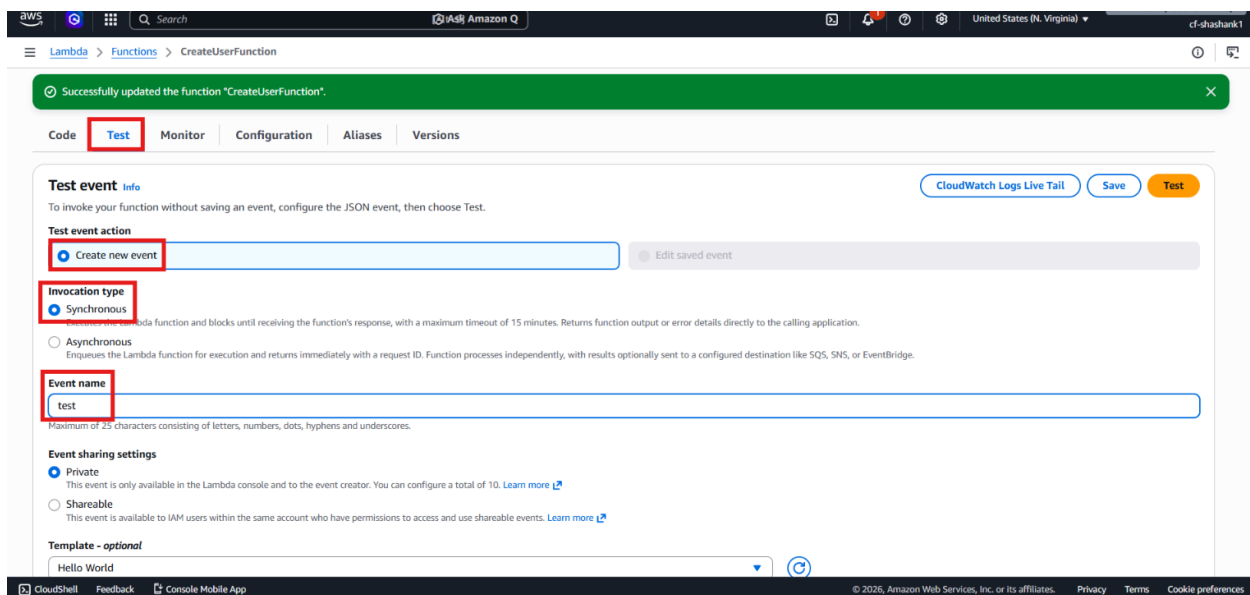
- Open the code editor.
- Replace the default handler code with the provided create-user Lambda code.
- Deploy the function.



- Confirm that the function was updated successfully.

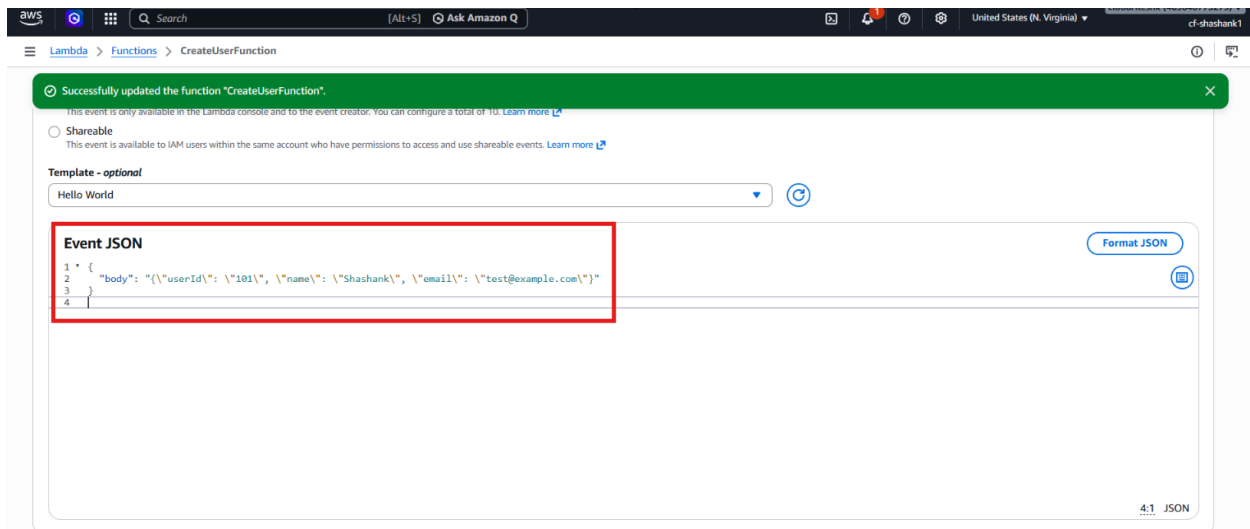
Step 3 – Test the Create User Function

- Open the **Test** tab.
- Click **Create new test event**.
- Enter a test event name.

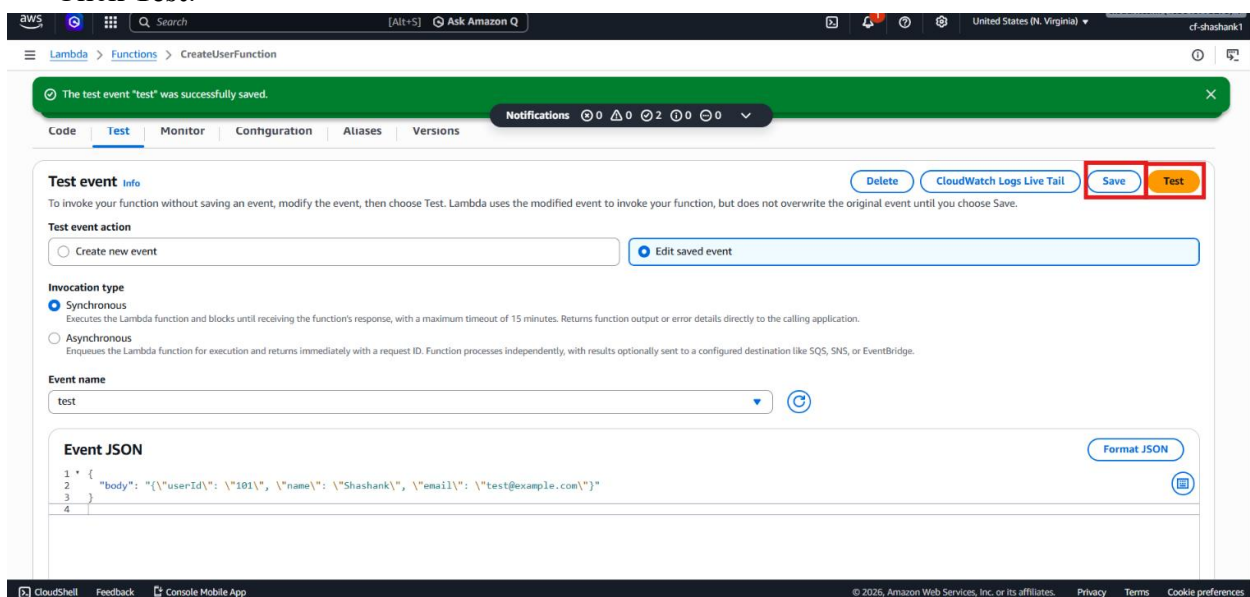


- Use an API Gateway–style payload with the body as a JSON string.

```
{
  "body": "{\"userId\": \"101\", \"name\": \"Shashank\", \"email\": \"test@example.com\"}"
}
```



- Click **Save**.
- Click **Test**.



- Verify that the function returns a success status.
- Open **Amazon DynamoDB**.
- Go to **Tables** → **UsersTable**.
- Click **Explore table items**.

UsersTable

Settings | Indexes | Monitor | Global tables | Backups | Exports and streams | Permissions

1 Protect your DynamoDB table from accidental writes and deletes
When you turn on point-in-time recovery (PITR), DynamoDB backs up your table data automatically so that you can restore to any given second in the preceding 1 to 35 days. Additional charges apply. [Learn more](#)

General information

Partition key: `userId (String)` | Sort key: `-` | Capacity mode: `On-demand` | Table status: `Active`

Alarms: `No active alarms` | Point-in-time recovery (PITR): `Off` | Item count: `0`

Average item size: `0 bytes` | Resource-based policy: `Not active`

Amazon Resource Name (ARN): `arn:aws:dynamodb:us-east-1:123456789012:table/UsersTable`

Read/write capacity: `Info`

- Confirm that the new record was inserted.

UsersTable

Autopreview | View table details

▼ Scan or query items

Scan | Query

Select a table or index: `Table - UsersTable` | Select attribute projection: `All attributes`

Filters - optional

Run | Reset

Completed - Items returned: 1 - Items scanned: 1 - Efficiency: 100% - RCUs consumed: 2

Table: UsersTable - Items returned (1)

Scan started on June 29, 2026, 21:32:56

userId (String)	email	name
101	test@exam...	Shashank

Step 4 – Create the Get User Function

- Return to **AWS Lambda**.
- Click **Create function**.
- Choose **Author from scratch**.
- Enter the function name: **GetUserFunction**
- Choose **Python 3.12**.

Create function [Info](#)

Choose one of the following options to create your function.

☒ **Author from scratch**
Start with a simple Hello World example.

☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container image**
Select a container image to deploy for your function.

Basic information

Function name
Enter a name that describes the purpose of your function.

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime [Info](#)
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Permissions
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose **Custom execution role** under **Additional settings**.

Custom settings [Info](#)

Durable execution - [new](#) [?](#) ☐
Build resilient, stateful applications with automatic failure recovery.

EC2 capacity provider - [new](#) [?](#) ☐
Run Lambda on your preferred EC2 instance types instead of Lambda's serverless compute (default).

- Under Custom Settings → Additional Settings, enable **ARM64 architecture** to use **x86_64**.

Custom settings [Info](#)

Durable execution - [new](#) [?](#) ☐
Build resilient, stateful applications with automatic failure recovery.

EC2 capacity provider - [new](#) [?](#) ☐
Run Lambda on your preferred EC2 instance types instead of Lambda's serverless compute (default).

▼ **Additional settings**
Customize your function architecture, execution role, HTTPS endpoints, tenant isolation mode, VPC, code signing, KMS key, and tags.

General [Info](#)

ARM64 architecture ☒
Use ARM64 instead of x86_64 (default).

Custom execution role ☐
Use custom execution role instead of new role with basic Lambda permissions (default).

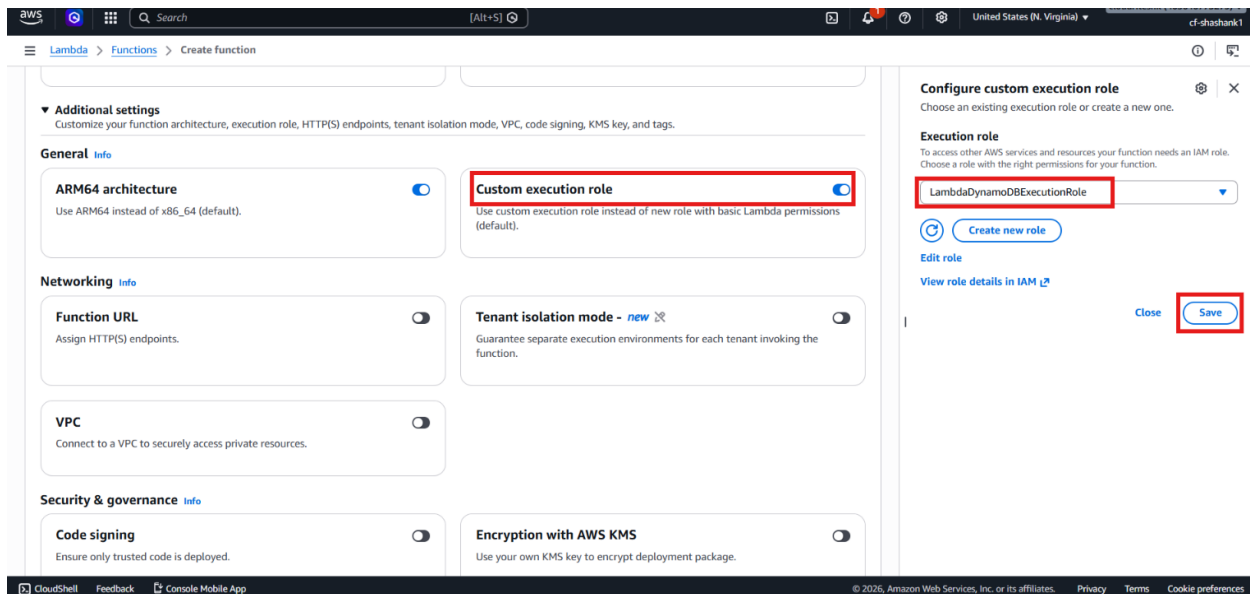
Networking [Info](#)

Function URL ☐
Assign HTTPS endpoints.

Tenant isolation mode - [new](#) [?](#) ☐
Guarantee separate execution environments for each tenant invoking the function.

VPC ☐
Connect to a VPC to securely access private resources.

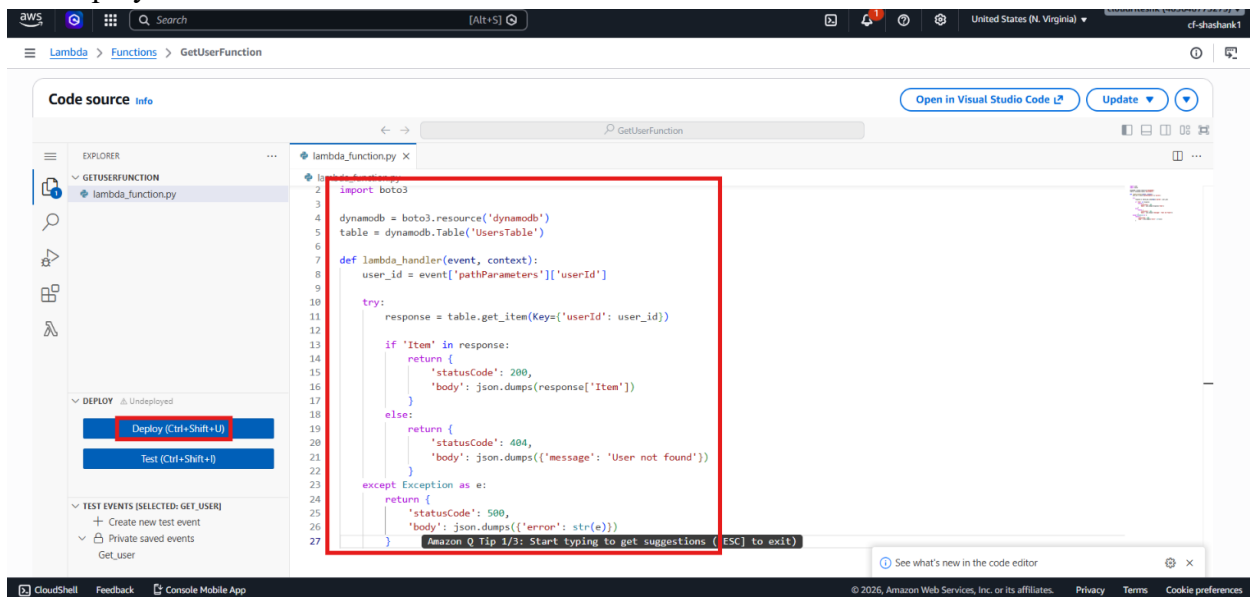
- Here Under **Additional Settings** click on **Custom Execution Role** and use the existing role **LambdaDynamoDBExecutionRole**.



- Click **Create function**.

Step 5 – Add the Get User Code

- Replace the default code with the provided get_user Lambda code.
- Deploy the function.



- Open the **Test** tab.

Successfully created the function "GetUserFunction". You can now change its code and configuration. To invoke your function with a test event, choose "Test".

Code **Test** Monitor Configuration Aliases Versions

Test event Info [CloudWatch Logs Live Tail](#) [Save](#) [Test](#)

To invoke your function without saving an event, configure the JSON event, then choose Test.

Test event action

☒ Create new event ☐ Edit saved event

Invocation type

☒ Synchronous
Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.

☐ Asynchronous
Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.

Event name

Get_user

Maximum of 25 characters consisting of letters, numbers, dots, hyphens and underscores.

Event sharing settings

☒ Private
This event is only available in the Lambda console and to the event creator. You can configure a total of 10. [Learn more](#)

☐ Shareable
This event is available to IAM users within the same account who have permissions to access and use shareable events. [Learn more](#)

Template - optional

Hello World

- Create a new test event using the path parameter structure.

```
{
  "pathParameters": {
    "userId": "101"
  }
}
```

Successfully created the function "GetUserFunction". You can now change its code and configuration. To invoke your function with a test event, choose "Test".

This event is only available in the Lambda console and to the event creator. You can configure a total of 10. [Learn more](#)

☐ Shareable
This event is available to IAM users within the same account who have permissions to access and use shareable events. [Learn more](#)

Template - optional

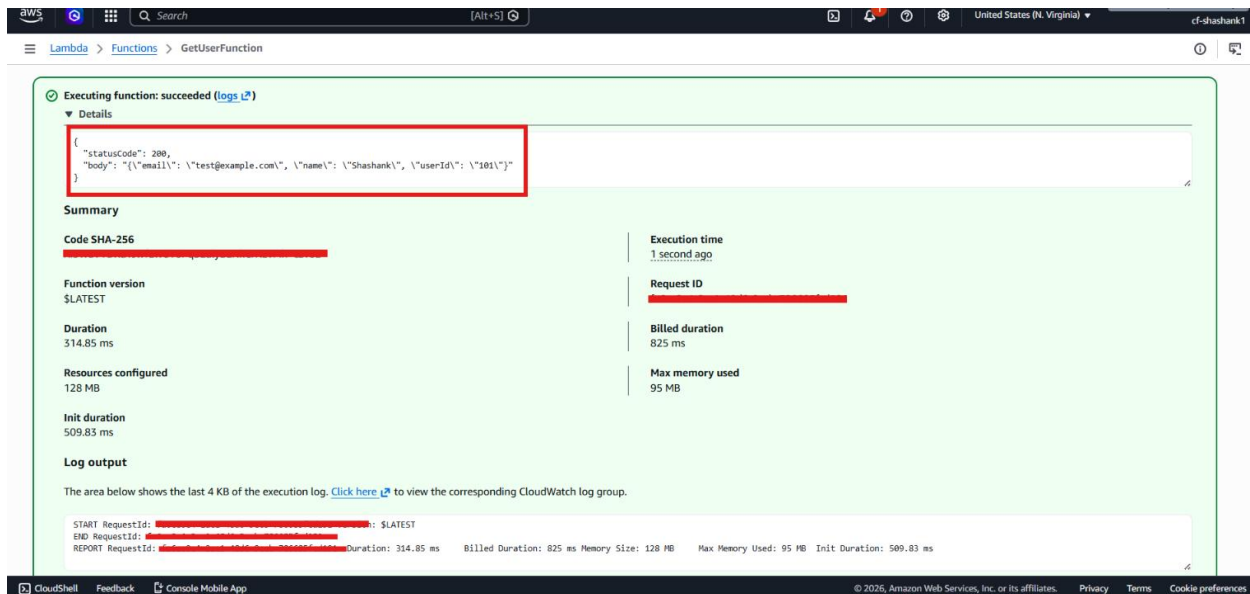
Hello World

Event JSON [Format JSON](#)

```
1 {
2   "pathParameters": {
3     "userId": "101"
4   }
5 }
```

6:1 JSON

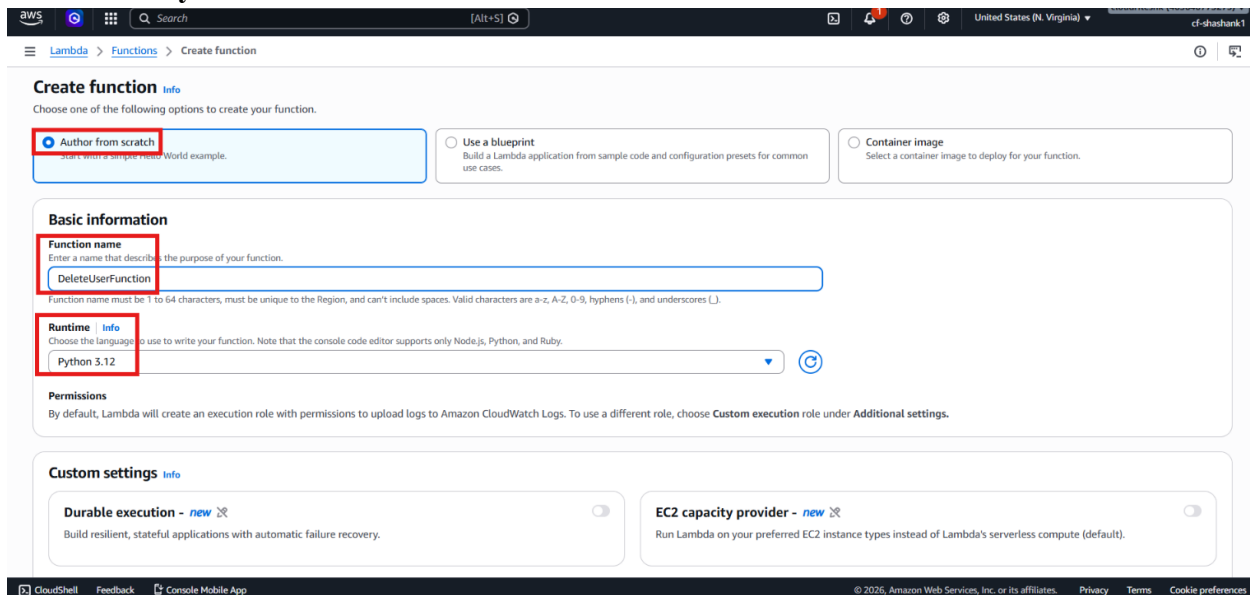
- Click **Save**.
- Click **Test**.



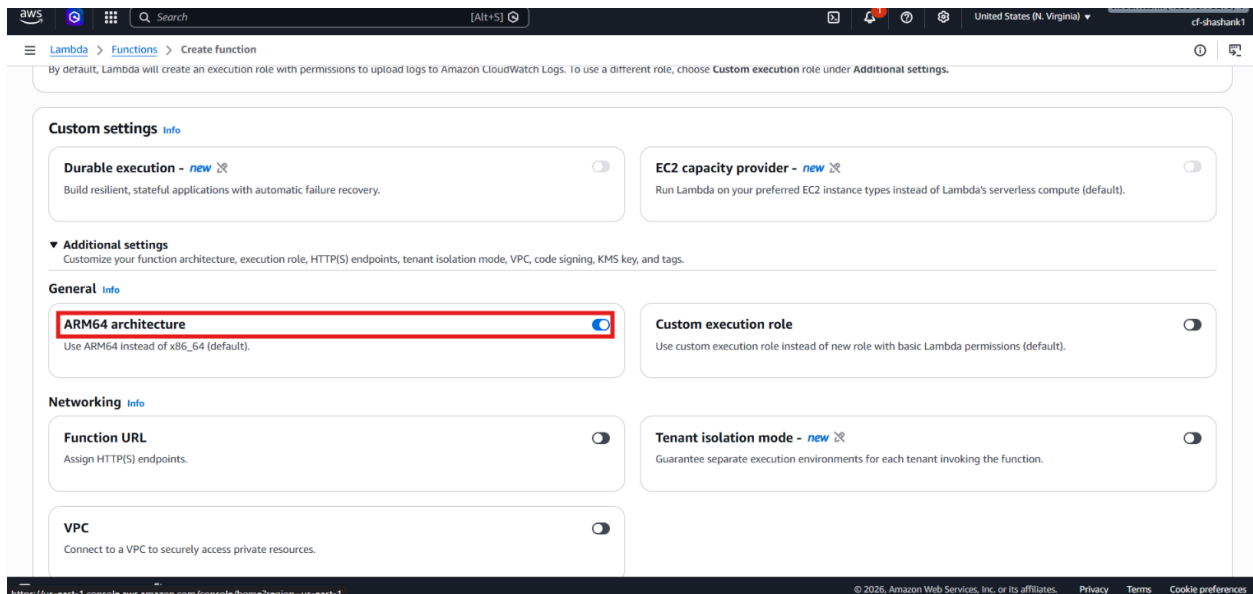
- Verify that the function returns the user details if the record exists.
- Change the userId to a missing value such as 102.
- Test again.
- Confirm that the function returns **User not found** for a missing record.

Step 6 – Create the Delete User Function

- Return to AWS Lambda.
- Click **Create function**.
- Choose **Author from scratch**.
- Enter the function name: DeleteUserFunction
- Choose **Python 3.12**.



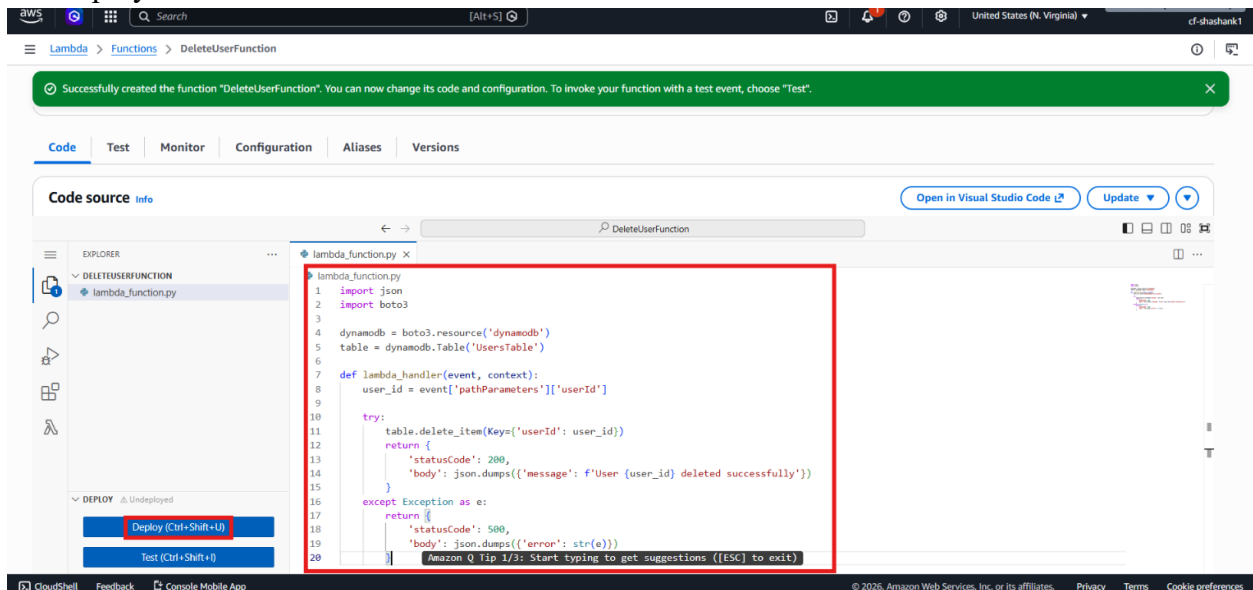
- Under Custom Settings → Additional Settings, enable **ARM64 architecture** to use **x86_64**.



- Here Under **Additional Settings** click on **Custom Execution Role** and use the existing role **LambdaDynamoDBExecutionRole**.
- Click **Create function**.

Step 7 – Add the Delete User Code

- Replace the default code with the provided delete-user Lambda code.
- Deploy the function.



- Open the **Test** tab.

Successfully updated the function "DeleteUserFunction".

Test event [Info](#)

To invoke your function without saving an event, configure the JSON event, then choose Test.

Test event action

☒ Create new event ☐ Edit saved event

Invocation type

☒ Synchronous
Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.

☐ Asynchronous
Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.

Event name

delete_user

Maximum of 25 characters consisting of letters, numbers, dots, hyphens and underscores.

Event sharing settings

☒ Private
This event is only available in the Lambda console and to the event creator. You can configure a total of 10. [Learn more](#)

☐ Shareable
This event is available to IAM users within the same account who have permissions to access and use shareable events. [Learn more](#)

Template - optional

Hello World

Event JSON [Format JSON](#)

- Create a new test event using the same path parameter structure.

```
{
  "pathParameters": {
    "userId": "101"
  }
}
```

Successfully updated the function "DeleteUserFunction".

Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.

☐ Asynchronous
Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.

Event name

delete_user

Maximum of 25 characters consisting of letters, numbers, dots, hyphens and underscores.

Event sharing settings

☒ Private
This event is only available in the Lambda console and to the event creator. You can configure a total of 10. [Learn more](#)

☐ Shareable
This event is available to IAM users within the same account who have permissions to access and use shareable events. [Learn more](#)

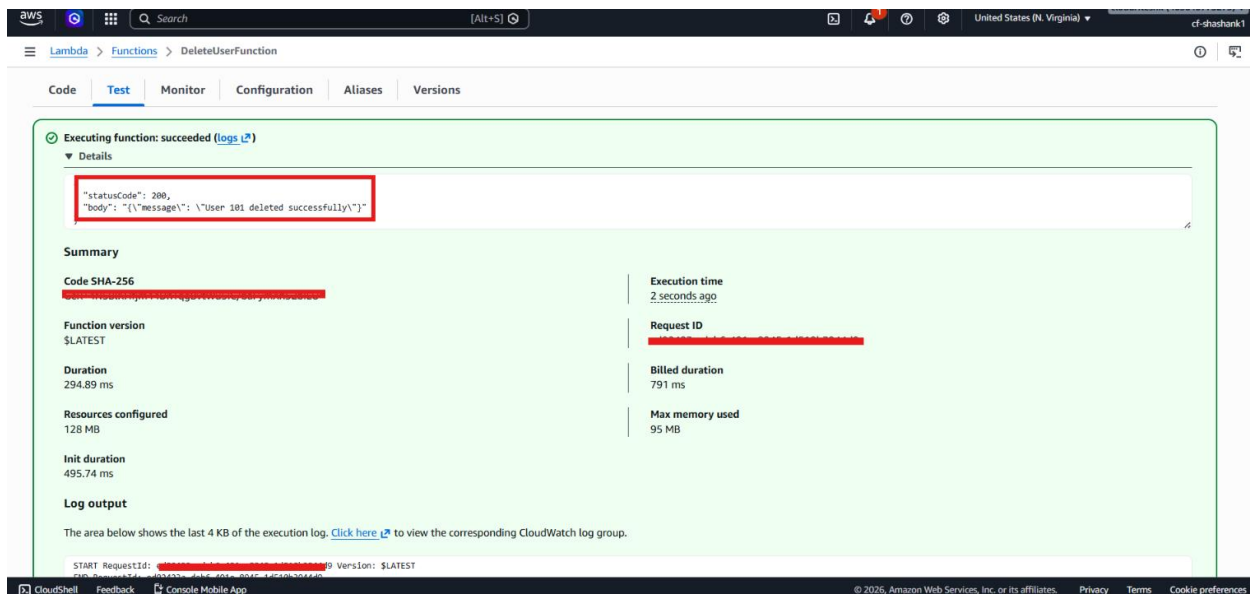
Template - optional

Hello World

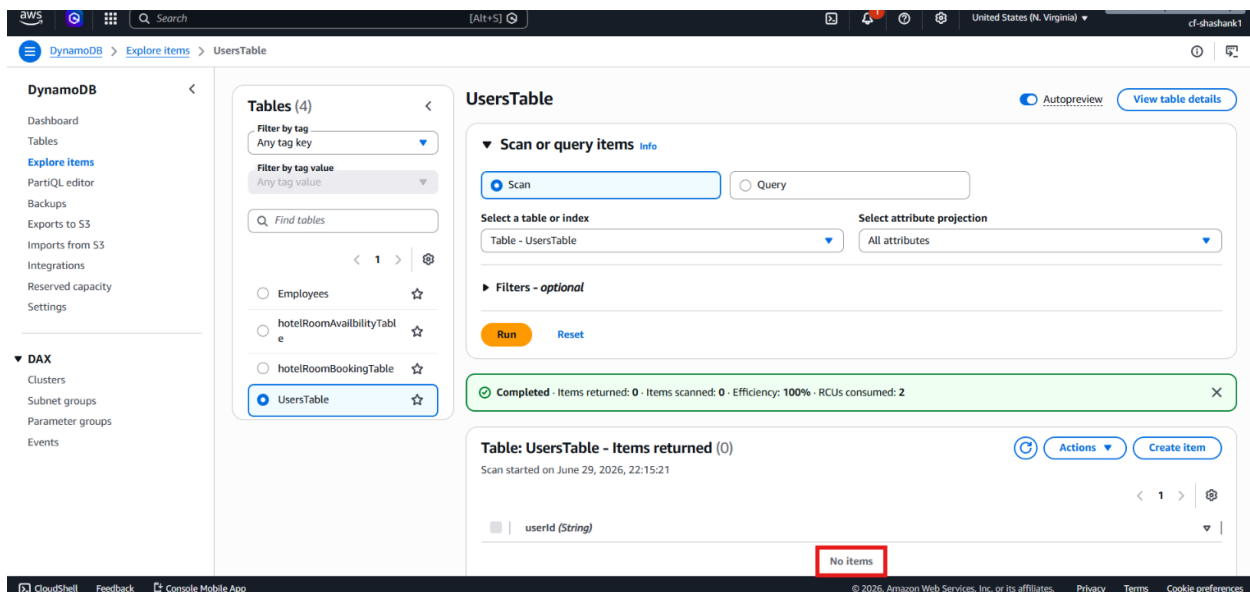
Event JSON [Format JSON](#)

```
1 {
2   "pathParameters": {
3     "userId": "101"
4   }
5 }
6
```

- Click **Save**.
- Click **Test**.



- Verify that the function returns a deletion success message.
- Open **Amazon DynamoDB**.
- Refresh **UsersTable**.



- Confirm that the record was removed.

Step 8 – Confirm the CRUD Backend Is Ready

- Verify that all three Lambda functions exist:
 - createUserFunction
 - getUserFunction
 - deleteUserFunction
- Confirm that all three functions use the same **IAM role**.

- Confirm that the DynamoDB table has been tested successfully with create, read, and delete operations.
- Prepare for **API Gateway** integration in the next part of the lab.

Verification

- The createUserFunction Lambda function inserted a new item into **UsersTable**.
- The getUserFunction Lambda function retrieved a record by userId.
- The deleteUserFunction Lambda function removed a record by userId.
- All Lambda functions used **LambdaDynamoDBExecutionRole**.
- The DynamoDB table was updated and verified successfully.
- The backend CRUD logic was completed and ready for **API Gateway**.